



Practical 1

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1 ALGORITHM

Bubble sort algorithm:-

* start with array size of n .

* take two loops.

* take first element in the first loop with " i " = index.

* enter into the second loop from $i+1$.

* compare both of them.

if $i > i+1$ swap.

else

continue

* At the end of second loop largest comes to last.

continue this process until no swaps needs end.

selection sort algorithm:-

* start with an array of n elements.

* take two loops one inside another.

* set the first element as current minimum.

* compare that with rest of the array.

* If a smaller is found update the minimum. After scanning swap

minimum with first element. move to next element and repeat process

continue until all elements are sorted. end.



Merge Sort algorithm:-

- 1) Divide the array into two halves.
- 2) Keep dividing until each part has one element.
- 3) Compare elements of the two parts.
- 4) Merge them in sorted order.
- 5) Repeat merging until the complete array is sorted.

Insertion Sort algorithm:-

- 1) Start with from second element.
- 2) Store it in key.
- 3) Compare key with the elements before it.
- 4) Shift larger element one position to right.
- 5) Insert key in its correct position.
- 6) Repeat until the array is sorted.

Quick Sort

- 1) Choose one element as a pivot.
- 2) Put smaller element on the left of pivot.
- 3) Put larger element on the right of pivot.
- 4) Apply the same process to right part. Continue until the array is sorted.

2 PROGRAM:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <algorithm> // for sort if needed
using namespace std;
// Helper to print array
void printArray(const vector<int> &arr) {
    for (size_t i=0; i<arr.size(); i++) {
        cout<<arr[i]<< " ";
    }
}
```

```
        cout<<"\n";
    }
    // Bubble Sort
    void bubbleSort(vector<int> &arr) {
        int n=arr.size();
        for (int i=0;i<n-1;i++)
            for (int j=0;j<n-i-1;j++)
                if (arr[j] > arr[j + 1])
                    swap(arr[j], arr[j + 1]);
    }
    // Selection Sort
    void selectionSort(vector<int> &arr) {
        int n = arr.size();
        for (int i = 0; i < n - 1; i++) {
            int min = i;
            for (int j = i + 1; j < n; j++)
                if (arr[j] < arr[min])
                    min = j;
            swap(arr[i], arr[min]);
        }
    }
    // Insertion Sort
    void insertionSort(vector<int> &arr) {
        int n = arr.size();
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
    // Merge Sort
    void merge(vector<int> &arr, int l, int m, int r) {
        int n1 = m - l + 1, n2 = r - m;
        vector<int> L(n1), R(n2);
        for (int i=0;i<n1;i++)L[i]=arr[l+i];
        for (int j=0;j<n2;j++) R[j]=arr[m+1+j];
        int i = 0, j = 0, k = l;
        while (i < n1 && j < n2) {
            if (L[i]<=R[j])arr[k++]=L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
    void mergeSort(vector<int> &arr, int l, int r) {
        if (l < r) {
            int m = (l + r) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m + 1, r);
            merge(arr, l, m, r);
        }
    }
}
```

```
// Quick Sort
int partition(vector<int> &arr,int low,int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int> &arr,int low,int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    const int n = 20; // smaller size for easy viewing
    vector<int> arr(n), temp;
    srand(time(0));
    for (int i = 0; i < n; i++) arr[i] = rand() % 100;
    cout << "Original Array:\n";
    printArray(arr);
    temp = arr;
    bubbleSort(temp);
    cout << "\nBubble Sort:\n";
    printArray(temp);
    temp = arr;
    selectionSort(temp);
    cout << "\nSelection Sort:\n";
    printArray(temp);
    temp = arr;
    insertionSort(temp);
    cout << "\nInsertion Sort:\n";
    printArray(temp);

    temp = arr;
    mergeSort(temp, 0, n - 1);
    cout << "\nMerge Sort:\n";
    printArray(temp);

    temp = arr;
    quickSort(temp, 0, n - 1);
    cout << "\nQuick Sort:\n";
    printArray(temp);
    return 0;
}
```

3 OUTPUT:

```
C:\WINDOWS\system32\cmd. x + v

Original Array:
72 71 94 58 58 86 8 11 12 58 4 54 10 78 21 62 86 81 82 84

Bubble Sort:
4 8 10 11 12 21 54 58 58 58 62 71 72 78 81 82 84 86 86 94

Selection Sort:
4 8 10 11 12 21 54 58 58 58 62 71 72 78 81 82 84 86 86 94

Insertion Sort:
4 8 10 11 12 21 54 58 58 58 62 71 72 78 81 82 84 86 86 94

Merge Sort:
4 8 10 11 12 21 54 58 58 58 62 71 72 78 81 82 84 86 86 94

Quick Sort:
4 8 10 11 12 21 54 58 58 58 62 71 72 78 81 82 84 86 86 94
```

4 TIME COMPLEXITY:

Bubble sort:-
time complexity:-

```
for (i=0; i<n-1; i++) { → n-1
    for (j=0; j<n-1; j++) { → n-i-1
        if (arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
        }
    }
}
```

$$T(n) = (n-1) + (n-2) + \dots + 1 \Rightarrow (n-1) + (n-2) + \dots + 1$$

$$= \frac{n(n-1)}{2} \Rightarrow O(n^2)$$



Time complexity! - (selection sort).

```
for (int i=0; i<n-1; i++) {
    int minIndex = i;
    for (int j=i+1; j<n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

total:-
 $n + (n-1) + (n-2) + \dots + 1$
 $\Rightarrow \frac{n(n+1)}{2} \Rightarrow O(n^2)$

merge sort.

Time complexity:-

```
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l+r)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}
```

merge sort recurrence:-

$$T(n) = 2T(n/2) + O(n).$$

compare with:-

$$T(n) = aT(n/b) + f(n)$$

$$a=2$$

$$b=2$$

$$f(n) = n$$

$$\log_2 2 = 1$$

$$k = \log_2 2$$

$$p=0$$

$$\text{So have } f(n) = \log_2 n$$

$$f(n) = (n \log n)$$

$$T(n) = O(n \log n)$$



Insertion sort ^{the} complexity:-
 for (int i=1; i<n; i++) { $\rightarrow n$
 let key = arr[i]; $\rightarrow 1$
 while (j>0 & arr[j]>key) { $\rightarrow i-1$
 arr[j+1] = arr[j]; $\rightarrow 1$
 j--;
 }
 arr[j+1] = key; } $\rightarrow 1$
Total:-
 $1+2+3+\dots+(n-1)$
 $\Rightarrow n(n-1)/2$
 $\Rightarrow O(n^2)$

array is sorted.
 void quicksort (int a[], int low, int high) {
 if (low < high) {
 int p = partition (a, low, high);
 quicksort (a, low, p-1);
 quicksort (a, p+1, high); } }
 partition = $O(n)$
 quicksort (left) + quicksort (right).

$$T(n) = 2T(n/2) + O(n)$$

By using master method.

$$a=2, b=2, f(n)=n, k=1, p=0.$$

$$\log_2 2 = 1$$

By comparing $1 < a \log_b$ value $k = \log_2 2$.

$$\therefore T(n) = n \log n$$

$$T(n) = n \log n \rightarrow \text{Best / merge case}$$

$$T(n) = O(n^2) \rightarrow \text{worst case}$$

5 CONCLUSION:

Bubble, Selection, and Insertion are slow for big data, good only for small or teaching.
 Merge and Quick are fast for large data, with Merge always safe and Quick usually fastest.